

Database Design Specification

AgentTrack AI: Refined 8-Collection Architecture (Updated)

1. Core Schemas & Architectural Justifications

The following database schemas reflect the exact requested architecture, refined to 8 distinct collections with the newly requested attributes added.

1. users

```
{
  "_id": "ObjectId",
  "name": "String",
  "email": "String",
  "password": "String", // hashed
  "role": "admin" | "manager" | "employee",
  "department": "String",
  "designation": "String",
  "isActive": "Boolean",
  "createdAt": "Date",
  "updatedAt": "Date"
}
```

Why it is needed:

Serves as the foundational identity table for Authentication & Authorization. It securely stores credentials (JWT integration) and enforces Role-Based Access Control (RBAC) across the platform.

2. projects

```
{
  "_id": "ObjectId",
  "title": "String",
  "description": "String",
  "managerId": "ObjectId", // Reference to users
  "priority": "low" | "medium" | "high",
  "startDate": "Date",
  "endDate": "Date",
  "status": "active" | "completed" | "on-hold",
  "progressPercentage": "Number",
  "createdAt": "Date",
  "updatedAt": "Date"
}
```

Why it is needed:

The parent container for organizing work. Storing high-level metrics like `progressPercentage` directly on the project document prevents the Manager Dashboard from executing heavy dynamic aggregations every time it loads.

3. employeeProjects

```
{
  "_id": "ObjectId",
  "projectId": "ObjectId", // Reference to projects
  "employeeId": "ObjectId", // Reference to users
  "assignedAt": "Date"
}
```

Why it is needed:

Acts as a normalized junction table. Because employees can work on multiple projects, and projects have multiple employees, this Many-to-Many relationship must be broken down to prevent BSON document size bloat that would occur if we embedded massive arrays of IDs directly into the users or projects collections.

4. tasks

```
{
  "_id": "ObjectId",
  "projectId": "ObjectId", // Reference to projects
  "title": "String",
  "description": "String",
  "assignedTo": "ObjectId", // Reference to users (Employee)
  "assignedBy": "ObjectId", // Reference to users (Manager)
  "priority": "low" | "medium" | "high",
  "status": "pending" | "in-progress" | "completed",
  "deadline": "Date",
  "estimatedHours": "Number",
  "actualHours": "Number",
  "completionPercentage": "Number",
  "createdAt": "Date",
  "updatedAt": "Date"
}
```

Why it is needed:

The granular unit of work required for Task Management. Important fields like `estimatedHours` and `assignedTo` are heavily utilized by the Workload Analysis Agent to calculate capacity and detect overloaded employees.

5. activityLogs

```
{
  "_id": "ObjectId",
  "userId": "ObjectId", // Reference to users
  "action": "PROJECT_CREATED" | "TASK_CREATED" | "TASK_UPDATED" |
  "TASK_COMPLETED",
  "entityType": "project" | "task",
  "entityId": "ObjectId",
  "details": "String",
  "createdAt": "Date"
}
```

Why it is needed:

Provides a comprehensive, system-wide audit trail. Without a dedicated taskUpdates collection, this single ledger is essential for the Reporting Agent to understand historical progress and generate accurate daily/weekly summaries by querying chronological events.

6. reports

```
{
  "_id": "ObjectId",
  "projectId": "ObjectId", // Reference to projects
  "reportType": "daily" | "weekly" | "project-summary",
  "summary": "String",
  "generatedAt": "Date"
}
```

Why it is needed:

Acts as a caching layer for the Reporting Agent. Large Language Models (LLMs) are computationally expensive; storing the generated summary ensures that when a manager opens the dashboard, the report loads instantly without re-triggering AI inferences.

7. risks

```
{
  "_id": "ObjectId",
  "projectId": "ObjectId", // Reference to projects
  "taskId": "ObjectId", // Reference to tasks
  "riskLevel": "low" | "medium" | "high",
  "reason": "String",
  "recommendation": "String",
  "resolved": "Boolean",
  "resolvedAt": "Date",
  "createdAt": "Date"
}
```

Why it is needed:

The dedicated output repository for the Risk Prediction Agent. By storing risks separately with a resolved boolean, the Manager Dashboard can easily query and display active alerts (e.g., overdue tasks) to facilitate immediate intervention.

8. workloads

```
{
  "_id": "ObjectId",
  "employeeId": "ObjectId", // Reference to users
  "totalAssignedHours": "Number",
  "activeTasksCount": "Number",
  "completedTasksCount": "Number",
  "status": "underutilized" | "optimal" | "overloaded",
  "aiRecommendation": "String",
  "calculatedAt": "Date"
}
```

Why it is needed:

The dedicated output/caching collection for the Workload Analysis Agent. Instead of dynamically aggregating every task's estimatedHours across the entire database every time the Manager Dashboard loads, this collection stores the AI's latest workload calculation and reassignment recommendations for each employee.

2. Entity-Relationship (ER) Diagram & Mapping

The following mapping details how data flows between the 8 distinct collections, ensuring referential integrity.

Entity / Collection	Relationship	Target Entity	Foreign Key / Join Condition
users (Manager)	1 : Many (1:N)	projects	projects.managerId → users._id
users (Employee)	Many : Many (M:N)	projects	Resolved via employeeProjects junction collection
projects	1 : Many (1:N)	tasks	tasks.projectId → projects._id
users	1 : Many (1:N)	tasks	tasks.assignedTo → users._id
projects	1 : Many (1:N)	reports	reports.projectId → projects._id
tasks	1 : 1 (or 1:N)	risks	risks.taskId → tasks._id
users	1 : Many (1:N)	activityLogs	activityLogs.userId → users._id
users (Employee)	1 : 1 (or 1:N)	workloads	workloads.employeeId → users._id

Architectural Layout

